National Institute of Technology Patna
Department of Electronics and Communication Engineering

Defensive KYC

Experiment Execution Plan

Must-Do Experiments — Step by Step

Authors:	Manmohan Vishwakarma (2304172) Ankur Raj (2304132)
Supervisor:	Dr. Manpuran Mahto, Dept. ECE
Programme:	B.Tech ECE, Minor Project (3rd Year)
Document:	Version 2.0 — April 2026

Purpose of this document. This plan defines, step-by-step, every experiment that *must* be completed before the minor project evaluation. It is structured to be followed directly by any team member independently, with a clear split of tasks that require GPU access versus those that can be run locally on an M2 Air. Results tables are left blank to be filled in as experiments are run.

Contents

1 Compute Strategy — Read This First **3**
- 1.1 Available Compute Environments . . . 3
- 1.2 Experiment-wise Compute Allocation . . . 3
- 1.3 Execution Strategy Under Limited GPU Availability . . . 3

2 Overview and Timeline **4**
- 2.1 Experiment Priority Classification . . . 4
- 2.2 4-Week Execution Timeline . . . 4

3 Phase 1 — Baseline Comparison (Week 1) **4**
- 3.1 Experiment 1 — Single-Model Baseline on FaceForensics++ . . . 5
  - 3.1.1 What You Are Measuring . . . 5
  - 3.1.2 Dataset . . . 5
  - 3.1.3 Step-by-Step Procedure . . . 5
  - 3.1.4 Expected Outputs . . . 5
  - 3.1.5 Pass / Fail Criterion . . . 6

4 Phase 2 — Module Validation (Weeks 1–3) **6**
- 4.1 Experiment 2 — S0 Stream Telemetry . . . 6
  - 4.1.1 What You Are Measuring . . . 6
  - 4.1.2 Data Collection . . . 6
  - 4.1.3 Step-by-Step Procedure . . . 6
  - 4.1.4 Expected Outputs . . . 7
- 4.2 Experiment 3 — S1 PRNU Sensor Forensics . . . 7
  - 4.2.1 What You Are Measuring . . . 7
  - 4.2.2 Data Collection . . . 7
  - 4.2.3 Step-by-Step Procedure . . . 8
  - 4.2.4 Expected Outputs . . . 8
- 4.3 Experiment 4 — S2 Moiré / Replay Attack Detection . . . 9
  - 4.3.1 What You Are Measuring . . . 9
  - 4.3.2 Data Collection . . . 9
  - 4.3.3 Step-by-Step Procedure . . . 9
  - 4.3.4 Expected Outputs . . . 10
- 4.4 Experiment 5 — S3 rPPG Biological Liveness Gate . . . 10
  - 4.4.1 What You Are Measuring . . . 10
  - 4.4.2 Critical Known Vulnerability — Must Be Tested . . . 10
  - 4.4.3 Data Collection . . . 10
  - 4.4.4 Step-by-Step Procedure . . . 11
  - 4.4.5 Expected Outputs . . . 11
- 4.5 Experiment 6 — Waterfall Pipeline Ablation Study . . . 11
  - 4.5.1 Ablation Variants . . . 11
  - 4.5.2 Step-by-Step Procedure . . . 12
  - 4.5.3 Expected Outputs . . . 12
- 4.6 Experiment 7 — Stage-wise Rejection Analysis . . . 12
  - 4.6.1 Step-by-Step Procedure . . . 12
  - 4.6.2 Expected Outputs . . . 13
- 4.7 Experiment 8 — Latency Breakdown . . . 13

4.7.1 Step-by-Step Procedure 13
4.7.2 Expected Outputs 14

5 Limitations to Report Honestly **14**

6 Summary Checklist **14**
6.1 Must-Do Experiments 15
6.2 Report Cleanup (Week 4) 15

Compute Strategy — Read This First

Key principle: Do not block progress on GPU availability. The majority of experiments can proceed immediately on an M2 Air. GPU is only required for deep learning training (Exp. 1, FTCA) and full-scale ablation inference (Exp. 6).

1.1 Available Compute Environments

- **Primary GPU System:** Tesla P100 (32 GB VRAM) — institutional access, may not always be available.
- **Local Development:** Apple M2 Air (unified memory, no discrete GPU) — always available, suitable for all signal-processing and lightweight inference.

1.2 Experiment-wise Compute Allocation

Experiment	M2 Air (Local)	GPU Required (P100)
Exp 1 — Xception baseline training	**No** (prohibitively slow)	**Yes**
FTCA training / fine-tuning	**No**	**Yes**
FTCA inference (small subset test)	Possible (slow)	Preferred
Exp 2 — Telemetry (S0)	**Yes**	No
Exp 3 — PRNU (S1)	**Yes** (BM3D is slow; use wavelet)	No
Exp 4 — Moiré / FFT (S2)	**Yes**	No
Exp 5 — rPPG (S3)	**Yes**	No
Exp 6 — Ablation (full dataset)	Reduced scale only	**Yes (preferred)**
Exp 7 — Stage-wise rejection	**Yes**	No
Exp 8 — Latency measurement	**Yes** (CPU path)	Yes (for S4 timing)

Table 1: Compute requirements per experiment. "Yes" means runnable locally without GPU.

1.3 Execution Strategy Under Limited GPU Availability

If institutional GPU access is unavailable or intermittent, adopt the following strategy:

- Complete Experiments 2–5 and 7–8 (CPU path) entirely on the M2 Air. Do not wait.
- Batch all GPU-dependent work (Exp. 1 training, FTCA inference, full Exp. 6) for when P100 access becomes available.

- For Exp. 6 on M2 Air, run a reduced test set (100 clips per class) to generate trend data, then scale up on GPU.
- Report CPU-only latency (Exp. 8) as a separate row — this is a useful result even without GPU data.

Overview and Timeline

2.1 Experiment Priority Classification

MUST-DO (Phases 1 & 2) — All 8 Experiments

These experiments are **non-negotiable**. Without them you cannot defend the core claims of the system. If time is very short, prioritise in order: Exp. 1, Exp. 5, Exp. 6, then remaining experiments.

2.2 4-Week Execution Timeline

Week	Dates (approx.)	Focus
Week 1	Training + 3 days	Exp. 1 (Baseline), Exp. 2 (Telemetry), Exp. 4 (Moiré)
Week 2	Days 8–14	Exp. 6 (Ablation), Exp. 8 (Latency)
Week 3	Days 15–21	Exp. 3 (PRNU), Exp. 5 (rPPG), Exp. 7 (Rejection %)
Week 4	Days 22–28	Robustness tests, report writing, presentation, buffer

Table 2: 4-week execution plan. Total experiments: 8 must-do.

Note on figures. Every figure in the architecture report marked *"simulated"* or *"replace with actual outputs"* **must** be replaced with real experimental output before submission.

Phase 1 — Baseline Comparison (Week 1)

MUST-DO — Experiment 1: Single-Model Baseline

Goal. Prove that FTCA outperforms a standard single-model detector. Without a baseline comparison, the novelty claim is indefensible.

Compute: GPU required (Tesla P100). Do not attempt full Xception training on M2 Air.

3.1 Experiment 1 — Single-Model Baseline on FaceForensics++

3.1.1 What You Are Measuring

Train a standard frame-level classifier (Xception or EfficientNet-B4) on FaceForensics++ and compare its AUC and EER directly against your FTCA model. Both models must be evaluated on *identical* test splits.

3.1.2 Dataset

- **FaceForensics++ (c23 compression)** — standard benchmark.
- Download: https://github.com/ondyari/FaceForensics
- Standard 720/140/140 train/val/test split.
- **Manipulation types:** Deepfakes, Face2Face, FaceSwap, NeuralTextures (all four).

3.1.3 Step-by-Step Procedure

Step 1. Prepare frames. Extract faces at 1 fps using `dlib` or `RetinaFace`. Resize to 224×224. Save as `.png`. Ensure balanced real/fake split.

Step 2. Train Xception baseline.

```
model = timm.create_model('xception', pretrained=True,
    num_classes=1)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.BCEWithLogitsLoss()
# Train 20 epochs, batch size 32, early stopping on val AUC
```

Use standard augmentations: random crop, horizontal flip, colour jitter. No test-time augmentation for the baseline.

Step 3. Evaluate baseline on FF++ test set. Record: AUC-ROC, EER, FAR at 1% FRR operating point, inference latency per frame.

Step 4. Evaluate FTCA on the identical test split (same frames, same splits).

Step 5. Record results in Table 3.

3.1.4 Expected Outputs

Model	AUC-ROC ↑	EER ↓	FAR@1%FRR ↓	Latency/frame
Xception (baseline)				
EfficientNet-B4 (optional)				
FTCA (ours)				

Table 3: Experiment 1 — Baseline comparison on FF++ c23.

3.1.5 Pass / Fail Criterion

Pass: FTCA AUC > Xception AUC by at least 1 percentage point on FF++ c23.
Concern: If FTCA AUC < 0.88 on FF++, check loss curves and dataloader before assuming the architecture is wrong.
Hard fail: If Xception beats FTCA — re-examine FTCA training before the presentation. Report this result honestly with explanation.

Phase 2 — Module Validation (Weeks 1–3)

These five experiments validate each pipeline stage individually. They are all must-do and can be run in parallel. **Experiments 2–5 and 7 run entirely on M2 Air** — do not wait for GPU access before starting them.

MUST-DO — Experiment 2: S0 Stream Telemetry Validation

Goal. Show a clear, measurable separation in the coefficient of variation (CV) of inter-frame timestamps between real webcam streams and virtual camera streams (OBS / ManyCam).

Compute: M2 Air (no GPU needed).

4.1 Experiment 2 — S0 Stream Telemetry

4.1.1 What You Are Measuring

$$\mathrm{CV} = \frac{\sigma(\Delta t)}{\mu(\Delta t)}$$

Real cameras exhibit CV $\approx 10^{-2}$; virtual cameras exhibit CV $< 10^{-4}$.

4.1.2 Data Collection

- **Real streams:** 10 clips (≥ 60 s each) from at least 3 different physical webcams / phone cameras. Log `time.perf_counter()` at each frame grab via `OpenCV VideoCapture`.
- **Virtual streams:** 10 clips from OBS Studio at 30 fps and 60 fps. Also test ManyCam if available.

4.1.3 Step-by-Step Procedure

Step 1. Record and log timestamps.

```
import cv2, numpy as np, time
cap = cv2.VideoCapture(0)  # or OBS virtual cam index
timestamps = []
while len(timestamps) < 600:  # ~20s at 30fps
    ret, _ = cap.read()
    if ret:
        timestamps.append(time.perf_counter())
deltas = np.diff(timestamps)
cv = deltas.std() / deltas.mean()
```

```
print(f"CV = {cv:.6f}")
```

Step 2. **Compute CV** for every recorded clip into `cv_real` and `cv_virtual`.

Step 3. **Plot histogram** of CV distributions (real vs. virtual) with rejection threshold $\varepsilon_{\text{tele}}$ as a vertical dashed line.

Step 4. **Choose threshold** $\varepsilon_{\text{tele}}$ as the midpoint between max real-CV and min virtual-CV. If distributions overlap, report the overlap as a known limitation.

Step 5. Record results in Table 4.

4.1.4 Expected Outputs

Source	Mean CV	Std CV	Correctly classified (%)
Real webcam (pooled)			
OBS 30 fps			
OBS 60 fps			
ManyCam (if available)			

Table 4: Experiment 2 — S0 Telemetry CV distributions.

MUST-DO — Experiment 3: S1 PRNU Forensics Validation

Goal. Show that PRNU fingerprint energy E_{PRNU} is significantly higher for real camera frames than for software-rendered frames. Characterise how compression degrades this signal.

Compute: M2 Air. Use wavelet denoiser (faster than BM3D) for initial results.

4.2 Experiment 3 — S1 PRNU Sensor Forensics

4.2.1 What You Are Measuring

$$E_{\text{PRNU}} = \|\hat{K}\|_F$$

averaged over $n = 30$ frames per clip.

4.2.2 Data Collection

- **Real cameras:** At least 3 different devices (laptop webcam, phone camera, DSLR if available). 5 clips per device, ≥ 30 frames each.
- **Synthetic / virtual sources:** OBS output, AI-generated video (FF++ fake set), WhatsApp-compressed real video, YouTube-downloaded talking-head clip.

4.2.3 Step-by-Step Procedure

Step 1. Extract noise residuals.

```
import numpy as np
from skimage.restoration import denoise_wavelet

def prnu_residual(frame_gray):
    denoised = denoise_wavelet(frame_gray, rescale_sigma=
       True)
    return frame_gray.astype(np.float32) - denoised

def prnu_energy(frames):
    residuals = [prnu_residual(f) for f in frames]
    K_hat = np.mean(residuals, axis=0)
    return np.linalg.norm(K_hat, ord='fro')
```

Step 2. Compute E_{PRNU} for every clip and source.

Step 3. Plot boxplots per source category.

Step 4. Choose threshold τ_{PRNU} minimising FAR + FRR on test set.

Step 5. If WhatsApp/YouTube compressed videos fall below τ_{PRNU}, document this as a known limitation (see Section 5).

4.2.4 Expected Outputs

Source	Mean E_{PRNU}	Std	Above threshold (%)
Real webcam			
Real phone camera			
OBS virtual cam			
AI-generated (FF++)			
WhatsApp compressed			
YouTube compressed			

Table 5: Experiment 3 — PRNU energy by source type.

MUST-DO — Experiment 4: S2 Moiré / Replay Detector

Goal. Demonstrate clean separation between a live camera feed and a screen-replayed feed using the Peak-to-Mean Ratio (PMR) of the 2D FFT magnitude spectrum. This should be your cleanest result.

Compute: M2 Air.

4.3 Experiment 4 — S2 Moiré / Replay Attack Detection

4.3.1 What You Are Measuring

$$\mathrm{PMR} = \frac{\max_{(u,v)\in\mathcal{R}} \tilde{M}_t(u,v)}{\mathrm{mean}_{(u,v)\in\mathcal{R}} \tilde{M}_t(u,v)}$$

4.3.2 Data Collection

- **Live feeds:** 10 clips of a real person talking to the webcam.
- **Phone screen replay:** Play a video on a phone screen, record with webcam. Test at 30 cm, 60 cm, 100 cm.
- **Monitor replay:** Play a video on a laptop/desktop screen.
- **High-resolution screen:** Test on a 4K display if available — this is your hardest case. Moiré may be weaker; document this.

4.3.3 Step-by-Step Procedure

Step 1. Compute 2D FFT and PMR per frame.

```
import numpy as np, cv2

def compute_pmr(frame_bgr, f_min=0.1, f_max=0.45):
    gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY).
       astype(np.float32)
    H, W = gray.shape
    F = np.fft.fftshift(np.fft.fft2(gray))
    M = np.log(1 + np.abs(F))
    u = np.fft.fftfreq(W)
    v = np.fft.fftfreq(H)
    U, V = np.meshgrid(u, v)
    R = np.fft.fftshift(np.sqrt(U**2 + V**2))
    mask = (R >= f_min) & (R <= f_max)
    region = M[mask]
    return region.max() / region.mean()
```

Step 2. Average PMR over all frames in each clip.

Step 3. Plot distributions as boxplots (live feed vs. phone vs. monitor).

Step 4. Include FFT visualisation: side-by-side of diffuse live-feed spectrum vs. spike-laden replay spectrum. Visually compelling for the presentation.

Step 5. Choose threshold τ_{PMR} and report classification accuracy.

4.3.4 Expected Outputs

Source	Mean PMR	Std	Detected as replay (%)
Live webcam feed			
Phone screen (30 cm)			
Phone screen (60 cm)			
Monitor replay			
4K display (if tested)			

Table 6: Experiment 4 — PMR scores by source type.

MUST-DO — Experiment 5: S3 rPPG Liveness Validation

Goal. Validate that the rPPG CHROM pipeline produces high-SNR pulse signals from real humans and near-zero SNR from deepfake videos. **This is your most fragile module.** Report limitations honestly.

Compute: M2 Air.

4.4 Experiment 5 — S3 rPPG Biological Liveness Gate

4.4.1 What You Are Measuring

$$\mathrm{SNR}_{\mathrm{pulse}} = 10 \log_{10} \left(\frac{\sum_{f \in B} |P(f)|^2}{\sum_{f \notin B} |P(f)|^2} \right)$$

where $B = [0.7, 4.0]\,\mathrm{Hz}$ is the valid cardiac band.

4.4.2 Critical Known Vulnerability — Must Be Tested

Replay of a real human video will contain the original rPPG signal. This attack *may* pass the rPPG gate. S2 (Moiré) is your defence against this. **You must test this case and report the result, whatever it is.**

4.4.3 Data Collection

- **Real humans:** 5 participants (ideally diverse skin tones), $\geq 30\,\mathrm{s}$ each, normal indoor lighting.
- **Deepfake videos:** 20 clips from FF++ fake set (Deepfakes and NeuralTextures types).
- **Replay of real human:** Record a participant, play the recording on a phone or monitor, record *that* with the webcam. Test whether the rPPG signal survives.

4.4.4 Step-by-Step Procedure

Step 1. Run CHROM pipeline. Verify MediaPipe Face Mesh ROI patches are correct by visualising landmarks on a test frame.

Step 2. Compute SNR for every clip.

Step 3. Plot SNR distributions as boxplots per category.

Step 4. Plot time-domain pulse waveform and PSD for at least one real human and one deepfake. This replaces Figure 4 (simulated) in your report.

Step 5. Test the replay case. If it passes, document: *"S3 does not defend against high-quality screen replay of real humans; S2 (Moiré) is the primary defence for this attack class."*

4.4.5 Expected Outputs

Scenario	Mean SNR (dB)	Std	Passes gate (%)
Real human (normal light)			
Real human (low light)			
Deepfake (FF++)			
Replay of real human			

Table 7: Experiment 5 — rPPG SNR by scenario.

MUST-DO — Experiment 6: Full Ablation Study

Goal. Prove that each pipeline module makes an independent, measurable contribution. This is the single most important experiment for evaluation and publication.

Compute: GPU preferred for full scale; M2 Air for reduced-scale preliminary results.

4.5 Experiment 6 — Waterfall Pipeline Ablation Study

4.5.1 Ablation Variants

V1. Baseline: S2 (Moiré/Replay detector) only.

V2. +Telemetry: V1 + S0.

V3. +PRNU: V2 + S1.

V4. +rPPG: V3 + S3.

V5. +FTCA: V4 + S4 (full FTCA model).

V6. Full system: V5 + score fusion (all modules, learned weights).

4.5.2 Step-by-Step Procedure

Step 1. Prepare a fixed test set. Must not overlap with training data. Include:

- Real videos (200 clips minimum)
- Screen replay attacks (50 clips)
- Virtual camera injections (50 clips — OBS recordings)
- Real-time deepfake / FF++ fakes (200 clips)
- Additional in-the-wild samples (YouTube / real-world clips, optional)

Step 2. For each variant V1–V6: run inference with only the specified modules active. Non-included modules should be *bypassed* (always return PASS).

Step 3. Compute AUC, FAR, FRR per variant on each attack category and on the full pooled test set.

Step 4. Fill Table 8.

4.5.3 Expected Outputs

Variant	AUC (FF++)	AUC (Real-world subset)	FAR (%)	FRR (%)	Latency (ms)	Notes
V1: S2 only						
V2: +S0						
V3: +S1						
V4: +S3						
V5: +S4						
V6: Full system						

Table 8: Experiment 6 — Ablation study results.

MUST-DO — Experiment 7: Stage-wise Rejection Analysis

Goal. Show what percentage of each attack class is caught at each stage. This demonstrates waterfall efficiency and is compelling for evaluators.

Compute: M2 Air.

4.6 Experiment 7 — Stage-wise Rejection Analysis

4.6.1 Step-by-Step Procedure

Step 1. Instrument the pipeline. Modify `core_engine.py` to log the rejection stage for every rejected stream.

Step 2. Run all attack types through the full pipeline.

Step 3. Compute per-stage rejection rates per attack class. These should sum to 100% if the system catches everything, or less if some attacks pass all stages.

Step 4. Plot as a stacked bar chart: attack type on x-axis, rejection percentage on y-axis, coloured by stage.

4.6.2 Expected Outputs

Attack Class	S0 (%)	S1 (%)	S2 (%)	S3 (%)	S4 (%)
Screen Replay					
Virtual Cam (OBS)					
Deepfake (FF++)					
Synthetic AI (Diffusion)					

Table 9: Experiment 7 — Percentage of attacks caught per stage.

MUST-DO — Experiment 8: Real Latency Measurement

Goal. Prove the claimed < 150 ms end-to-end latency with real measurements.

Compute: M2 Air for CPU path; GPU for S4 timing (report separately).

4.7 Experiment 8 — Latency Breakdown

4.7.1 Step-by-Step Procedure

Step 1. Instrument each stage.

```
import time

def timed(fn, *args):
    t0 = time.perf_counter()
    result = fn(*args)
    return result, (time.perf_counter() - t0) * 1000  # ms
```

Step 2. Run 100 clips through the full pipeline. Use 1080p input, 16-frame window.

Step 3. Report mean ± std per stage and cumulative end-to-end latency (median, 95th percentile, max).

Step 4. Also test: CPU-only mode (disable S4) and report as a separate row.

4.7.2 Expected Outputs

Stage	Hardware	Mean (ms)	Std (ms)	Cumulative (ms)
S0: Telemetry	CPU			
S1: PRNU	CPU			
S2: Moiré	CPU			
S3: rPPG	CPU			
S4: FTCA	P100 GPU			
Fusion + output	CPU			
Total (GPU path)				
Total (CPU-only)				

Table 10: Experiment 8 — Per-stage and end-to-end latency breakdown.

Limitations to Report Honestly

State these limitations explicitly in the presentation and report. Doing so proactively is a sign of research maturity; hiding them invites uncomfortable questions.

L1. Replay of real human (Experiment 5). A high-quality screen replay of a real person may pass S3 (rPPG). S2 (Moiré) is the primary defence; for 4K displays this defence may also weaken. Hardware-level camera attestation remains an open research problem.

L2. PRNU under compression (Experiment 3). Heavy compression (WhatsApp, YouTube) degrades PRNU fingerprint energy. In real deployment, video is always compressed, reducing S1 effectiveness. Mitigation: S0 already rejects most virtual cameras before S1 is reached.

L3. rPPG in low light and for darker skin tones. rPPG SNR degrades in low-light conditions and is known to perform worse on darker skin tones due to lower optical reflectance. This is a known bias in the literature and must be acknowledged explicitly.

L4. FTCA not adversarially trained. A sophisticated adversary with white-box knowledge of the FTCA architecture could potentially craft adversarial examples. The system is designed for operational KYC, not white-box adversarial settings.

L5. Evaluation set size. The real-world test set is limited in size relative to established benchmarks. Statistical confidence intervals on AUC should be reported (bootstrap or DeLong's method). This does not invalidate the contribution but should be stated.

Summary Checklist

6.1 Must-Do Experiments

- ☐ **Exp 1:** Xception baseline trained and evaluated on FF++ c23 *[GPU]*
- ☐ **Exp 1:** FTCA evaluated on identical FF++ c23 test split *[GPU]*
- ☐ **Exp 1:** Table 3 filled with real numbers
- ☐ **Exp 2:** Real webcam and OBS timestamps logged (10 clips each) *[M2 Air]*
- ☐ **Exp 2:** CV histogram generated (Figure for report) *[M2 Air]*
- ☐ **Exp 2:** Threshold $\varepsilon_{\text{tele}}$ chosen and documented
- ☐ **Exp 3:** PRNU energy computed for ≥ 3 real devices + OBS + AI video *[M2 Air]*
- ☐ **Exp 3:** PRNU boxplot generated
- ☐ **Exp 3:** WhatsApp / YouTube compression case tested
- ☐ **Exp 4:** Live feed vs. phone replay vs. monitor replay recorded *[M2 Air]*
- ☐ **Exp 4:** PMR distribution plot generated
- ☐ **Exp 4:** Side-by-side FFT visualisation image created
- ☐ **Exp 5:** rPPG run on ≥ 5 real humans *[M2 Air]*
- ☐ **Exp 5:** rPPG run on 20 FF++ deepfake clips
- ☐ **Exp 5: Replay of real human tested (critical)**
- ☐ **Exp 5:** Figure 4 in report replaced with real pulse/PSD data
- ☐ **Exp 6:** Fixed test set assembled (real + all attack types) *[GPU preferred]*
- ☐ **Exp 6:** All 6 ablation variants evaluated
- ☐ **Exp 6:** Table 8 filled
- ☐ **Exp 7:** Pipeline instrumented to log rejection stage *[M2 Air]*
- ☐ **Exp 7:** Stage-wise rejection chart generated
- ☐ **Exp 8:** Per-stage latency measured on P100 (100 clips) *[GPU + M2 Air]*
- ☐ **Exp 8:** CPU-only latency measured
- ☐ **Exp 8:** Table 10 filled

6.2 Report Cleanup (Week 4)

- ☐ All figures marked "simulated" replaced with real experimental output
- ☐ All result tables filled with real numbers
- ☐ Limitations section added to the report
- ☐ References verified (especially the "no existing KYC product deploys rPPG" claim)
- ☐ GitHub repository code matches the architecture described in the report

Final Note

This document is a living plan. Update result tables in-place as experiments are run and note any deviations from expected behaviour. Unexpected negative results — PRNU breaking under compression, rPPG passing a replay — are *not* failures. They are findings. Document them clearly and explain how the broader pipeline architecture mitigates them. Evaluators reward honesty and systems thinking far more than they reward perfect numbers.